Evolutionary Graph Neural Networks

Anonymous authors

Abstract

Graph neural networks (GNNs) learn low-dimensional embeddings of nodes in large graphs for predictive analysis. Existing approaches are primarily designed for static (attributed) graphs or preserving node proximity of temporal patterns. Unfortunately, they cannot accurately forecast *change of node attributes* and *birth and death of links*. How to infer embeddings for predicting future snapshots remains an open problem. It poses great challenges to GNN design, since the network architecture should model the *co-evolution* of node attributes and graph structure in dynamic attributed graphs. Here we present a novel framework of evolutionary graph neural networks to generate node embedding series from the seen graph series, preserve the impact of early graphs on the current graph, and infer future node embeddings. It employs a multi-task loss function based on attribute inference and link prediction over time. Our algorithm outperforms strong baselines on predicting an entire unseen snapshot of personal attributes and interpersonal links in evolving graphs.

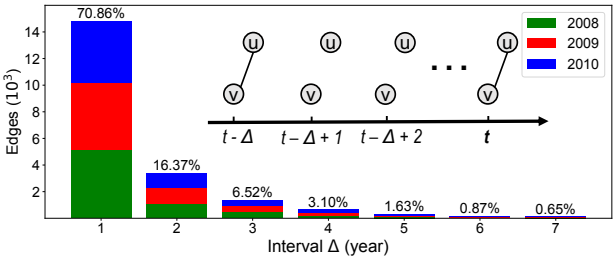
Introduction

Real-world graphs evolve over time. For example, social network users constantly establish and/or remove links between each other via behaviors of following, unfollowing, mentioning, replying, etc. The user's node attributes such as textual features from generated content are also constantly changing. On academic co-authorship networks, researchers are looking for collaborations all through their career. Their research topics or interests may change due to new collaborative projects. So, the co-evolutionary patterns of node attribute and graph structure are complex yet valuable, and need to be learned in the predictive model, if we aim at forecasting future attributes and structures.

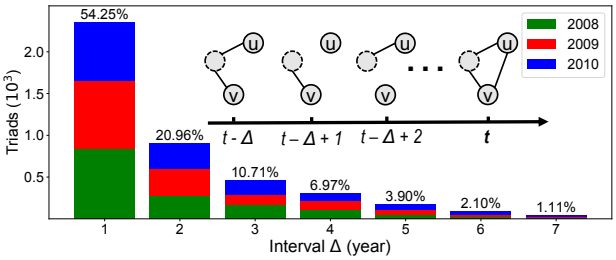
Graph neural networks (GNNs) have widely been used to learn low-dimensional representations of nodes from graph data for many tasks such as node classification, community detection, and link prediction. Bruna *et al.* (2013) introduced convolution operations on graph. Kipf *et al.* (2016) proposed GCN models to approximate the graph Laplacian spectrum. Hamilton *et al.* (2017a) proposed GRAPHSAGE which transited from spectral convolution to spatial-based aggregation operation aggregating node features from neighbor samples. However, the majority of existing GNN models assumed the graph structures as well as node attributes are static. To be more specific, the input of model is a single attributed graph. We can merge all the graph snapshots into one and feed into these models, but the temporal dimension would be collapsed and the evolutionary information would be lost. Therefore, those models cannot learn the co-evolutionary patterns of node attribute and graph structure information. Designing a new GNN framework for modeling evolutionary graph patterns is desired and remains an open problem.

The problem is nontrivial as modeling co-evolutionary

Copyright © 2020, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.



(a) If the links did exist in previous graphs, more than 29% were at least two time points before ($\Delta \geq 2$).



(b) If the links could be generated by closing a triad, more than 45% of the triads occurred at least two time points before ($\Delta \geq 2$).

Figure 1: The formation of a new link in co-authorship networks depends on more than one previous graphs. Repeating a link and/or closing a triad more than two time points before can be commonly seen in the link formation. Modeling the dependency is necessary for preserving evolutionary patterns in node embeddings and predicting future graphs.

graph patterns imposes great challenges. First, the conventional convolution or aggregation operation along the axis of graph structural depth would not meed the demand. In evolutionary graphs, the node attribute and graph structure of current graph snapshot depends on not only its current state or the last state, but also the states of multiple previous graphs. For example, in an evolutionary co-authorship network, the formation of a future collaboration link between two authors can be related to their co-authorship relations 2 or 3 or even 5 years ago. In Figure 1a, we visualize the process of two author nodes developing a future link at $t \in \{2008, 2009, 2010\}$ if they were linked at $t - \Delta$. We present the proportion of these links according to their corresponding interval Δ: though a fair amount of the links occurred in the very recent year ($\Delta = 1$), more than 29% of new links can be traced back to previous years of $\Delta > 1$. In Figure 1b, we visualize another fundamental mechanism of link formation–triad closure (Coleman and Coleman 1994). And it is evident that almost 46% links that were formed through this process fell in the interval of $\Delta > 1$. In other words, previous graph states beyond the very recent one still contain valuable information for predicting the future. Multiple previous snapshots of the evolutionary graphs should be fused into node representation learning.

Second, node attributes and graph structures are co-evolving and mutually influencing each other. Again, take the co-authorship networks as an example. Forming a new link (i.e., a new collaboration) can absolutely extend the research topics and improve the impact of the authors. And having new research topics or having improvement on h-index can help an author form new collaborations. Existing methods were not able to learn effective node embeddings for simultaneously forecasting node attributes and graph structures. Both types of information must be used to supervise the GNN models for evolutionary embedding learning.

To address the challenges, we propose a novel framework "Evolutionary Graph Neural Networks" (EvoGNN) that can effectively capture the co-evolution of node attribute and graph structure. It learns the impacts of previous graph snapshots on the current one as transformation matrices. It generates node embedding series from the seen graph series, and has the ability of inferring node embeddings for completely unseen future graph (see Figure 2a). At each of the K layers, EvoGNN first aggregates the neighbors' representations and concatenates with the node's self representations. The concatenated embedding is used to infer the node embedding at time t. We design a K-layer "highway" network architecture which multiplies node embeddings by the corresponding transformation matrices, i.e., $\mathbf{W}^{(k)}$, and finally sums into the set of graph node embeddings at t.

To obtain the ability of forecasting a completely unseen future graph snapshot, EvoGNN employs a multi-task loss function based on the tasks of attribute inference and link prediction over time (see Figure 2b). On one hand, the node representations at each training time would be transformed back into the raw attribute space by multiplying the transformation matrix $\mathbf{M}$. We define an attribute evolutionary loss as the discrepancy between distributions of the attribute prediction and observation at each time point. On the other hand, the node representations at each training time also get evaluated by a random walk based structural evolutionary loss. With these two types of loss terms, the co-evolution of node attribute and graph structure is explicitly learned and used for forecasting node embeddings.

We empirically evaluate the effectiveness of EvoGNN on forecasting an entire future snapshot of personal attributes and co-authorship links in evolutionary graphs from public academic data. Experimental results show that it can outperform competitive baselines by $+3.7\%$ on F1 score (bigger is better) for link prediction, and by -47.6% on RMSE (smaller is better) for attribute inference.

Related work

Graph neural networks. The convolution operation for graph from spectral domain using the graph Laplacian matrix was first proposed in (Bruna et al. 2013). An extension of a simple max/mean pooling at the beginning of the network to reduce the cost of graph Fourier transform operation was proposed in (Henaff, Bruna, and LeCun 2015). In (Kipf and Welling 2016), the authors proposed the spectral-based GCN model for semi-supervised node classification on graph. PATCHY-SAN (Niepert, Ahmed, and Kutzkov 2016) generalized the convolution operation to multiple graphs using graph labeling procedure to assign a unique order of nodes. R-GCN (Schlichtkrull et al. 2018) extended the idea of GCN on knowledge graphs by training different weights for different relation types. GraphSAGE (Hamilton, Ying, and Leskovec 2017a) was designed to learn node embeddings of graph in either supervised or unsupervised fashion. It is the first inductive learning method on graph which aggregates attribute information from the sampled neighborhood of a node. PinSAGE (Ying et al. 2018) later adapted the the former one to a web-scale recommender system. In (Xu et al. 2018), the authors provided theoretical explanations on the power of graph neural networks. A comprehensive review on recent methods and applications of representation learning on graphs was provided in (Hamilton, Ying, and Leskovec 2017b). All these models assume a single static graph structure and node attributes as input. They cannot be directly applied on evolutionary graph.

Dynamic graph representation learning. Dynamic network embedding methods aim at modeling the temporal pattern of graph topological structure. CTDNE (Nguyen et al. 2018) incorporated temporal structure dependencies into embeddings of continuous-time dynamic networks by conducting temporal random walks. DynamicTriad (Zhou et al. 2018) preserved the dynamic structural information in a network by modeling the triadic closure process. DySAT (Sankar et al. 2018) employed a self-attention mechanism which considers both neighbor nodes and the historical representations. These methods did not handle node attribute. They can neither capture the evolution pattern of node attribute nor forecast future node attribute information.

Some deep graph neural methods explored the idea of combining GNN with recurrent neural network (RNN) for leaning node embeddings in dynamic attributed network. WD-GCN (Manessi, Rozza, and Manzo 2017) stacked a LSTM (Hochreiter and Schmidhuber 1997) module on top

Capability	CTDNE	DYNAMICTRIAD	DYSAT	WD/CD-GCN	GCRN	DCRNN	STGCN	EVOLVEGCN	**EVOGNN**
	Method								
Evolution of G	✓	✓	✓	✓	✓			✓	✓
Evolution of $\mathbf{X}$				✓	✓	✓	✓	✓	✓
Forecasting G	✓	✓	✓					✓	✓
Forecasting $\mathbf{X}$						✓	✓		✓

Table 1: Capability of dynamic methods on (i) capturing the evolution of graph structure G, (ii) capturing the evolution of node attributes $\mathbf{X}$, (iii) forecasting future graph structure G, and (iv) forecasting future node attributes $\mathbf{X}$. Our proposed method EVOGNN meets requirements in all four aspects.

of a GCN (Kipf and Welling 2016) module while CD-GCN added an additional skip connection. The first architecture is similarly adopted by GCRN (Seo et al. 2018) and the authors also proposed a modified LSTM by replacing fully connected layers with graph convolution (Defferrard, Bresson, and Vandergheynst 2016). These methods need to be trained under the supervision of explicit node/graph labels and cannot forecast future graph structure or node attribute.

Another set of approaches for spatiotemporal traffic forecasting aim at capturing the evolutionary pattern of node attribute given a fixed graph structure. DCRNN (Li et al. 2017) modeled the traffic flow as a diffusion process on a directed graph and adopted an encoder-decoder architecture for capturing the temporal attribute dependencies. STGCN (Yu, Yin, and Zhu 2017) modeled the traffic network as a general graph and employed a fully convolutional structure (Defferrard, Bresson, and Vandergheynst 2016) on time axis. These methods assume the graph structure remains static, thus being incapable of capturing the evolution of graph structure or forecasting into future graph structure. Recently, EVOLVEGCN (Pareja et al. 2019) was proposed for representation learning on dynamic attributed graph. Different from previous methods, EVOLVEGCN used the RNN to evolve the graph model itself over time instead of the node embeddings generated by the GNN model on each graph snapshot. This model can be used to forecast graph structure but not for node attribute.

The capability of these dynamic methods and our proposed method EVOGNN is summarized in Table 1. To the best of our knowledge, our proposed method EVOGNN is the first model that can capture the co-evolution of graph structure and node attribute and also can forecast graph structure and node attribute simultaneously. We will compare EVOGNN against models capable of forecasting graph structure and/or future node attribute in experiments.

Methodology

In this section, we first formally define the research problem and describe the preliminaries. Then we present the design of EVOGNN including evolutionary embedding aggregation and a joint evolutionary loss. The core idea is to forecast the future node attribute value and network structure by learning their co-evolution from previous network snapshots.

Problem definition

A static graph is represented as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of nodes and $\mathcal{E}$ denotes the set of edges. The node attribute matrix of G is represented as $\mathbf{X} \in \mathbb{R}^{n \times \hat{d}}$, where each row $\mathbf{x}_v$ describes the $\hat{d}$-dimensional raw attribute vector of node v. A evolutionary graph can be represented by multiple graph snapshots $(G^0, \mathbf{X}^0), (G^1, \mathbf{X}^1), ..., (G^T, \mathbf{X}^T)$ across different time steps. The research problem of evolutionary graph representation learning is: given a dataset of evolutionary graph $\mathcal{D} = \{(G^0, \mathbf{X}^0), (G^1, \mathbf{X}^1), ..., (G^T, \mathbf{X}^T)\}$, we aim at learning a function $f_{evolve}(\mathcal{D}) : \mathcal{V} \rightarrow \mathbb{R}^d (\mathcal{V} = \bigcup_{t=0}^{T} \mathcal{V}^t)$ that can map each node $v \in \mathcal{V}$ into a d-dimensional representation $\mathbf{h}_v^t$ for each time point $t \in \{0, 1, ..., T\}$.

Preliminaries

Static graph embedding models learn $f_{static}(G, \mathbf{X}) : \mathcal{V} \rightarrow \mathbb{R}^d$ which maps each node v into a d-dimensional hidden representation $\mathbf{h}_v$. The node embedding matrix $\mathbf{H} \in \mathbb{R}^{n \times d}$ should incorporate the information in both the node attribute matrix $\mathbf{X}$ and the graph structure G. Earlier spectral methods like GCN (2016) define f_{static} as a convolution operation:

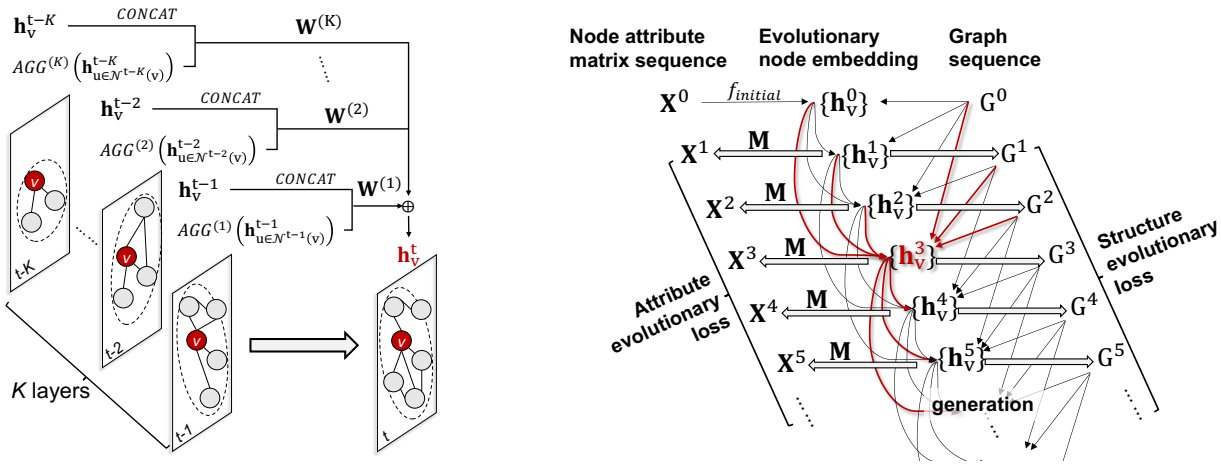
$$\mathbf{H}^l = \sigma(\hat{\mathbf{A}}\mathbf{H}^{l-1}\mathbf{\Theta}), \tag{1}$$

where $\hat{\mathbf{A}} = \mathbf{I} + \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}$. $\mathbf{A}$ is the adjacent matrix of G, and $\mathbf{\Theta}$ are the model parameters. Note here, the superscript l refers to the convolutional structure depth in static graph G. Spatial methods like GRAPHSAGE (2017a) defines f_{static} by an aggregation operation:

$$\mathbf{h}_{\mathcal{N}(v)}^l = \text{AGG}(\{\mathbf{h}_u^{l-1}, \forall u \in \mathcal{N}(v)\}), \tag{2}$$

which aggregates attribute information from node v's sampled neighbors $\mathcal{N}(v)$ and then concatenates with node self representation: $\mathbf{h}_v^l = \sigma\left(\mathbf{W}^l \cdot \text{CONCAT}(\mathbf{h}_v^{l-1}, \mathbf{h}_{\mathcal{N}(v)}^l)\right)$, where $\mathbf{W}^l$ is the weight matrix at depth l to be learned. However, these methods are designed for static graph and do not consider the evolutionary pattern of node attribute value and graph structure over time. Conceptually, f_{static} blends a single pair of $(G, \mathbf{X})$ into $\mathbf{H}$. So, directly applying f_{static} when the attributed graph G is actively evolving would fail to capture the valuable temporal trend revealed in the process.

For evolutionary graph, like the hidden representation matrix generated by f_{static}, $\mathbf{H}^t \in \mathbb{R}^{n \times d}$ should embed the node attribute information in $\mathbf{X}^t$ plus the graph structure information in G^t. Furthermore, $\mathbf{H}^t$ should also embed the influence from multiple previous graph hidden representations $\mathbf{H}^{t-1}$, $\mathbf{H}^{t-2}$, ..., $\mathbf{H}^{t-K}$. So, the key questions are: (1) how to fuse the influence of multiple previous hidden node representations into the current one; and (2) how to effectively model the co-evolution process of node attributes $\mathbf{X}^t$ and graph structure G^t exhibits in $\mathcal{D}$ along the temporal axis. We propose a novel framework EVOGNN for learning



(a) Evolutionary embedding generation

(b) Evolutionary attribute and structure losses (K = 3)

Figure 2: Visual illustration of EvoGNN's embedding generation and evolutionary loss functions.

node representations on evolutionary graph. It has a K-layer "highway" network architecture to answer the first question, and employs a multi-task loss function based on attribute inference and link prediction over time to answer the second question. Next, we introduce EvoGNN's embedding generation process, evolutionary loss, and training process.

Proposed method: EvoGNN

In this section, we present the evolutionary embedding generation process in EvoGNN model. It is illustrated in Figure 2(a). The pseudo code is presented in Algorithm 1. Assuming EvoGNN's weight matrices $\mathbf{W}^{(k)}, \forall k \in \{1, ..., K\}$ are learned and fixed, the inputs into the algorithm include: the snapshot series of the evolving graph $G^t = (\mathcal{V}, \mathcal{E}^t), \forall t \in \{0, 1, ..., T\}$, the initial node attribute matrix $\mathbf{X}^0$, and the model evolution depth K. For initialization, we use a function $f_{initial}$ to transform the leading snapshot of graph $(G^0, \mathbf{X}^0)$ into hidden representations $\mathbf{H}^0 \in \mathbb{R}^{n \times d}$. We can also use a separate EvoGNN model ($K = 1$) in this case or other existing f_{static} methods such as node2vec (Grover and Leskovec 2016) or GraphSAGE (Hamilton, Ying, and Leskovec 2017a), as the initialization rolls back into the task of unsupervised node embedding learning on a static graph. Other inputs of the algorithm include: aggregator functions $\text{AGG}^{(k)}, \forall k \in \{1, ..., K\}$ (e.g., nonparametric element-wise mean aggregator), non-linear function σ (e.g., $ReLU$), and graph neighborhood function $\mathcal{N}^t$ that is essentially determined by G^t.

After the initialization (Line 1 in Algorithm 1), EvoGNN generates hidden node representations along time points $t = 1 \ldots T$ in a cascade mode (outer loop from Line 2 to 7). At a particular time point t (the inner loop from Line 3 to 6), for each node v, neighbor node representations in the last K or precisely $\min(t, K)$ time points are aggregated as $\text{AGG}^{(k)}(\{\mathbf{h}_u^{t-k}, \forall u \in \mathcal{N}^{t-k}(v)\})$ at the end of Line 4. Then, they get concatenated with the node v's self representation $\mathbf{h}_v^{t-k}$ at the time point $(t - k)$. The concatenated representation $\text{CONCAT}\big(\mathbf{h}_v^{t-k}, \text{AGG}^{(k)}(\{\mathbf{h}_u^{t-k}, \forall u \in \mathcal{N}^{t-k}(v)\})\big), \forall k \in \min(t, K)$ encodes v's attribute and structure information before time point t, and serve for generating v's representation at t. After this, we apply L2 normalization on the node embeddings (Line 5).

K-layer "highway" network architecture. At the heart of the EvoGNN model is the K-layer "highway" network architecture which fuses the influence from previous K graph snapshots on the node representations in the current snapshot. It multiplies the previous node representations by the transformation matrix $\mathbf{W}^{(k)}$ (where k is the time difference between the previous snapshot and the current one) and takes the summation. Note that the superscript k in $\text{AGG}^{(\cdot)}$ and $\mathbf{W}^{(\cdot)}$ also acts as the depth (i.e., layer's order) in the graph neural network model. If we set $K = 1$, EvoGNN can be seen as a model placed in the Markov property (Qu, Bengio, and Tang 2019) where each node embedding matrix $\mathbf{H}^t$ depends solely on the previous node embedding matrix $\mathbf{H}^{t-1}$. However, this is a strong assumption that does not always hold in real cases. An example is that, in an evolutionary co-authorship network, authors collaborate in one year does not necessarily indicate they will collaborate in the next year; but authors could be more likely to collaborate if they collaborate and publish papers in the last few years. By leveraging this K-layer "highway" (from $\mathbf{H}^{t-1}$, $\mathbf{H}^{t-2}$, ... to $\mathbf{H}^{t-K}$), EvoGNN is able to fuse influence from multiple previous graph snapshots for learning the current node embeddings. At the same time, it maintains the flexibility of choosing a proper K for specific data or applications. We will discuss the choice of K in the experiments.

Inferring future node embeddings. The output of EvoGNN is a series of node hidden representations $\mathbf{H}^t, \forall t \in \{1, ..., T\}$ that summarizes the input evolutionary attributed graph. In addition, an arbitrary number of node hidden representations at future time points (say, $\mathbf{H}^{T+1}$, $\mathbf{H}^{T+2}$, ...). The future node embeddings directly reflect EvoGNN's forecasting capability on the temporal trend

Algorithm 1: EvoGNN evolutionary embedding generation

Input : Evolutionary graph $G^t = (\mathcal{V}, \mathcal{E}^t), \forall t \in \{0, 1, ..., T\}$; initial node attributes $\mathbf{x}_v^0, \forall v \in \mathcal{V}$ and a transformation function $f_{initial}$; evolution depth K; weight matrices $\mathbf{W}^{(k)}$ and aggregator functions $\text{AGG}^{(k)}, \forall k \in \{1, ..., K\}$; function of non-linearity σ; and, graph G^t's neighborhood function $\mathcal{N}^t : v \to 2^{\mathcal{V}}, \forall t \in \{0, 1, ..., T\}$

Output: Node hidden vector representations $\mathbf{h}_v^t$ for all $v \in \mathcal{V}$ and $t \in \{0, 1, ..., T\}$

$\mathbf{h}_v^0 \leftarrow f_{initial}(\mathbf{x}_v^0), v \in \mathcal{V}$;
for $t = 1 \ldots T$ **do**
  for $v \in \mathcal{V}$ **do**
    $\mathbf{h}_v^t \leftarrow \sigma\left(\sum_{k=1}^{\min(t,K)} \mathbf{W}^{(k)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{t-k}, \text{AGG}^{(k)}(\{\mathbf{h}_u^{t-k}, \forall u \in \mathcal{N}^{t-k}(v)\})\right)\right)$;
    $\mathbf{h}_v^t \leftarrow \mathbf{h}_v^t / \|\mathbf{h}_v^t\|_2$
  end
end

of node attributes and graph structure beyond the observed graph time series. Forecasting into far future would be really challenging. In this paper, we only consider predicting node embeddings of the next time point (time $t+1$) beyond the training evolutionary graph (to time t). We leave forecasting multiple time steps as future work.

Learning with multi-task evolutionary loss

In this section, we describe the training process of EvoGNN according to the overall loss function defined in Eqn. (3).

$$\min_{\mathbf{h}_v^t, v \in \mathcal{V}, t \in \{1,...,T\}} \mathcal{J} = \sum_{t=1}^{T} \sum_{v \in \mathcal{V}} \left(\mathcal{J}_{\mathbf{X}^t}(\mathbf{h}_v^t) + \mathcal{J}_{G^t}(\mathbf{h}_v^t)\right). \tag{3}$$

In order to learn the model on the evolutionary attributed graph for forecasting, we apply this multi-task loss function on generating node hidden representations $\mathbf{h}_v^t$ for all $v \in \mathcal{V}$ and over training time points $t \in \{1, ..., T\}$. For each forward propagation round, each $\mathbf{H}^t, \forall t \in \{1, ..., T\}$ gets evaluated by the loss. Then we use stochastic gradient descent to update the set of weight matrices $\mathbf{W}^{(k)}, \forall k \in \{1, ..., K\}$ which parameterizes the EvoGNN model.

The multi-task evolutionary loss function is composed of two parts. One is the attribute evolutionary loss $\mathcal{J}_{\mathbf{X}^t}$, and the other is structure evolutionary loss $\mathcal{J}_{G^t}$. Specifically, the attribute loss $\mathcal{J}_{\mathbf{X}^t}$ evaluates the performance on node attribute inference. It transforms node hidden representation back into the raw attribute space and encourages the resulting vector to approximate the node's true attribute distribution. And, the graph structure loss $\mathcal{J}_{G^t}$ evaluates on graph link prediction. It encourages nodes with observed edge between them to have similar hidden representations and makes disconnected nodes to have distinct representations. Figure 2 (b) visualizes an example of the multi-task evolutionary loss process ($K = 3$). Next, we provide more details on the two loss terms.

Attribute evolutionary loss for attribute inference The attribute loss $\mathcal{J}_{\mathbf{X}^t}$ is defined as below:

$$\mathcal{J}_{\mathbf{X}^t}(\mathbf{h}_v^t) = \|\mathbf{M} \cdot \mathbf{h}_v^t - \mathbf{x}_v^t\|_F^2, \tag{4}$$

where $v \in \mathcal{V}, t \in \{1, 2, \ldots, T\}$. Given a node hidden representation $\mathbf{h}_v^t \in \mathbb{R}^d$, the attribute transformation matrix $\mathbf{M} \in \mathbb{R}^{\hat{d} \times d}$ is used for mapping $\mathbf{h}_v^t$ back into the raw attribute space. The node v's remapped attribute inference vector $\mathbf{M} \cdot \mathbf{h}_v^t \in \mathbb{R}^{\hat{d}}$ is then compared against the true node attribute vector $\mathbf{x}_v^t \in \mathbb{R}^{\hat{d}}$. We use L2 norm to measure the discrepancy between the two distributions. Note that just like the weight matrices $\mathbf{W}^{(k)}$, $\mathbf{M}$ also gets updated with back propagation. $\mathbf{M}$ is irrelevant to the depth K and only describes the mapping from hidden representation space into raw attribute space across time.

Structure evolutionary loss for link prediction The structure evolutionary loss $\mathcal{J}_{G^t}$ is defined as below:

$$\begin{aligned} \mathcal{J}_{G^t}(\mathbf{h}_v^t) \quad = \quad & -\log\left(\sigma({\mathbf{h}_v^t}^\top \mathbf{h}_u^t)\right) \\ & -Q \cdot \mathbb{E}_{u' \sim P_n(v)} \log\left(\sigma(-{\mathbf{h}_v^t}^\top \mathbf{h}_{u'}^t)\right), \end{aligned} \tag{5}$$

where $v \in \mathcal{V}, t \in \{1, 2, \ldots, T\}$. Node u is one of the 1st-order neighbors of node v. This can be generalized as that node u co-occurs near node v on a fixed-length random walk. Node u' is a negative node, i.e., disconnected node with v, drawn according to the negative sampling distribution $P_n(v)$. Q is the number of negative samples and σ is the non-linear function. Intuitively, Eqn. (5) pulls similar nodes closer and pushes dissimilar nodes away in the hidden space. Taken together with Eqn. (4), the multi-task evolutionary loss function Eqn. (3) captures the co-evolution of node attribute and graph structure over time.

Experiments

We evaluate EvoGNN on two forecasting tasks: (1) node attribute prediction and (2) graph link prediction. Forecasting the *entire* piece of future, instead of completing partial missing information at a current time, needs the understanding of evolutionary patterns. In all experiments, we test the model effectiveness on predicting the *next graph snapshot*. We first describe the datasets and experimental settings. Then we provide and analyze the results.

Datasets

We collected $226,611$ papers published from 2001 to 2010 in computer science from Microsoft Academic Graph. We

Table 2: EvoGNN performs better than GNN baselines on forecasting node attributes in $\mathbf{X}^{2010}$ and links in G^{2010}.

	$\mathcal{D}^{(2K)}$					$\mathcal{D}^{(10K)}$				
	Attributes $\mathbf{X}^{2010}$		Links in G^{2010}			Attributes $\mathbf{X}^{2010}$		Links in G^{2010}		
Method	MAE	RMSE	AUC	F1	P@50, 100, 200	MAE	RMSE	AUC	F1	P@50, 100, 200
GraphSAGE	0.643	1.265	0.084	0.201	0.38, 0.44, 0.41	0.729	1.438	0.039	0.078	0.36, 0.40, 0.42
DynamicTriad	N/A	N/A	0.112	0.241	0.76, 0.62, 0.60	N/A	N/A	0.058	0.147	0.60, 0.59, 0.57
DySAT	N/A	N/A	0.120	0.222	0.54, 0.46, 0.38	N/A	N/A	0.036	0.127	0.48, 0.43, 0.36
DCRNN	0.458	0.960	0.019	0.073	0.12, 0.10, 0.10	0.423	0.853	0.006	0.027	0.09, 0.06, 0.03
STGCN	0.478	1.127	0.006	0.027	0.04, 0.02, 0.04	0.567	1.589	0.001	0.007	0.04, 0.04, 0.02
EvolveGCN	0.684	1.279	0.133	0.256	0.78, 0.80, 0.67	0.768	1.603	0.069	0.161	0.69, 0.74, 0.59
EvoGNN-Min	0.495	1.086	0.102	0.218	0.52, 0.54, 0.52	0.472	0.913	0.050	0.121	0.48, 0.46, 0.52
EvoGNN-NMF	**0.449**	**0.945**	0.138	0.260	**0.80**, 0.82, 0.66	**0.420**	0.841	0.072	0.162	0.78, 0.81, **0.63**
EvoGNN-Evo	0.451	0.947	**0.142**	**0.265**	**0.80**, **0.83**, **0.70**	0.421	**0.840**	**0.076**	**0.167**	**0.81**, **0.84**, 0.62

built a series of ten co-authorship graphs, each graph for one year. We ranked the authors by number of papers and used the top $2,000$ and $10,000$ to make two evolutionary graph datasets $\mathcal{D}^{(2K)}$ and $\mathcal{D}^{(10K)}$. We used the venues and the title's words of the author's papers as node attribute. We filtered out infrequent venues and words. We have 316 venues and $3,549$ words in $\mathcal{D}^{(2K)}$; and 448 venues, $6,442$ words in $\mathcal{D}^{(10K)}$. More statistical descriptions of the datasets can be found in the Supplemental Materials.

Experimental settings

Competitive methods: We consider two EvoGNN variants w.r.t. choices of initialization function $f_{initial}$. In addition, we consider an ablated version by removing the K-layer "highway" structure.

- EvoGNN-NMF: Non-negative matrix factorization (Févotte and Idier 2011) was applied on $\mathbf{X}^0$ for generating the initial $\mathbf{H}^0$;
- EvoGNN-Evo: A separate EvoGNN model was used on the first graph snapshot $(G^0, \mathbf{X}^0)$ to generate $\mathbf{H}^0$;
- EvoGNN-Min: Setting $K = 1$ so that each $\mathbf{H}^t$ depends solely on the previous $\mathbf{H}^{t-1}$ (uses itself for $\mathbf{H}^0$).

We compare EvoGNN against related GNNs in Table 1.

- GraphSAGE: We test 3 variants based on the range of training graph snapshots and report the best performance.
- DynamicTriad and DySAT: These two dynamic network embedding methods cannot handle node attribute. All graphs are fed for training, and we focus on the task of *future graph link prediction*.
- DCRNN and STGCN: For training, we use all node attribute matrices and the most recent graph. The final prediction matrix is used for the *future node attribute prediction* task. The node embeddings outputted by the diffusion convolutional layer, or the spatio-temporal convolutional block, are used for the *future graph link prediction* task.
- EvolveGCN: All graph snapshots are provided as input of this model. We use the link prediction loss for training, and use the node embeddings outputted by the evolving graph convolution unit for *future graph link prediction*.

We merge multiple graph snapshots into single one as input into GraphSAGE variants. And we compute the attribute inference matrix as $\mathbf{H}^{t_{test}} \cdot (\mathrm{tr}(\mathbf{H}^{t_{train}}) \cdot \mathbf{X}^{t_{train}})$, where $\mathrm{tr}(\,)$ denotes transpose operator, for GraphSAGE and EvolveGCN. For EvoGNN, we conduct random walks as the neighborhood function $\mathcal{N}$ with sampling sizes $S = 20$, length of random walk $L = 2$, and number of negative samples $Q = 200$. More details on the settings of baseline methods *as well as* parameter and training settings can be found in the Supplementary Materials.

Evaluation metrics: For future node attribute prediction, we use Mean Average Error (MAE) and Root Mean Squared Error (RMSE) as evaluation metrics; for future graph link prediction, we use Area Under the precision-recall Curve (AUC), F1 measure, and Precision@$50, 100, 200$.

Results

Table 2 presents experimental results and we have the following observations. First, among all variants of GraphSAGE, the one trained using all historical graph snapshots makes the lowest MAE, RMSE, and the highest AUC and F1. However, simply merging and feeding all previous graph snapshots into a static GNN loses co-evolutionary patterns and thus cannot accurately forecast node attribute and graph structure. Our ablated EvoGNN-Min model (without the highway structure) is able to reduce RMSE to 1.086 (-14.2% relatively) and increases F1 score to 0.218 ($+8.5\%$ relatively) on $\mathcal{D}^{(2K)}$ (-36.5% on RMSE and $+55.1\%$ on F1 on $\mathcal{D}^{(10K)}$). This demonstrates the importance of capturing the co-evolutionary patterns.

Second, both dynamic network embedding methods DynamicTriad and DySAT give comparable performance to EvoGNN for predicting future links in G^{2010}. DynamicTriad is able to outperform EvoGNN-Min in terms of AUC and F1 on two datasets. However, these two methods are designed for only capturing the dynamics of evolving graph structure, whereas EvoGNN aims at modeling the co-evolution of node attribute and graph structure. The "N/A" values in the table indicates they are not applicable for predicting future node attribute. Furthermore, by fusing information from multiple previous states, EvoGNN-Evo can have F1 scores of 0.265 and 0.167 on the two datasets, which is $+10.0\%$ and $+13.6\%$ relatively over DynamicTriad.

Table 3: Forecasting node attributes in $\mathbf{X}^{2010}$ and links in G^{2010} with different values of model depth K.

	$\mathcal{D}^{(2K)}$					$\mathcal{D}^{(10K)}$				
	Attributes $\mathbf{X}^{2010}$		Links in G^{2010}			Attributes $\mathbf{X}^{2010}$		Links in G^{2010}		
Depth	MAE	RMSE	AUC	F1	P@50, 100, 200	MAE	RMSE	AUC	F1	P@50, 100, 200
$K = 1$	0.456	0.960	0.116	0.237	0.72, 0.74, 0.66	0.425	0.850	0.071	0.152	0.60, 0.66, 0.64
$K = 2$	0.452	0.949	0.134	0.255	**0.82**, 0.82, 0.69	**0.420**	0.841	0.074	0.163	0.80, 0.81, **0.63**
$K = 3$	**0.451**	**0.947**	**0.142**	**0.265**	0.80, **0.83**, **0.70**	0.421	**0.840**	**0.076**	**0.167**	**0.81**, **0.84**, 0.62
$K = 4$	0.457	0.963	0.117	0.234	0.72, 0.68, 0.58	0.423	0.848	0.039	0.148	0.52, 0.56, 0.55
$K = 5$	0.534	1.086	0.086	0.200	0.59, 0.58, 0.50	0.431	0.857	0.030	0.144	0.50, 0.52, 0.56

This demonstrates the advantage of the proposed highway structure in EvoGNN.

Third, for spatio-temporal forecasting methods DCRNN and STGCN, they can model the evolutionary pattern of node attribute, but they cannot effectively forecast graph structure. DCRNN makes RMSEs of 0.960 and 0.853 on the two datasets, but only scores F1s of 0.073 and 0.027. They assume that the graph structure remains static over time. In contrast, EvoGNN-Evo scores RMSEs of 0.947 and 0.840, -1.4% and -1.5% relatively over DCRNN. When predicting future links in G^{2010}, EvoGNN-Evo significantly outperforms DCRNN and makes $+263\%$ improvements of F1 scores. This reflects our design of EvoGNN is powerful: we assume node attribute and graph structure evolve over time and use the co-evolutionary patterns for forecasting.

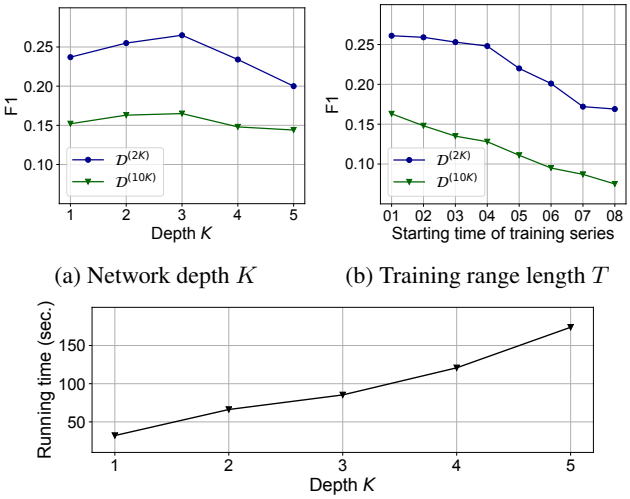
Fourth, the most competitive EvolveGCN has the best performance for predicting links among all baselines. It makes F1s of 0.256 and 0.161 and EvoGNN-Evo scores 0.265 ($+3.5\%$ relatively) and 0.167 ($+3.7\%$ relatively). Despite the input of all historical graph snapshots are the same for EvolveGCN and our EvoGNN model, one fundamental difference between them is that EvolveGCN assumes the underlying force driving the graph evolution origins from the changes in graph structure, instead of the mutual impact between node attribute and graph structure. In other words, it does not model the node attribute evolution. Thus, EvolveGCN only focuses on link prediction and can only generate attribute predictions of similar quality as the static model GraphSAGE. As a result, EvoGNN-Evo outperforms it by -26.0% and -47.6% in terms of RMSE.

We also found the performance of EvoGNN-NMF and EvoGNN-Evo are similar though EvoGNN-Evo is slightly better for predicting future links. This demonstrates the robustness of EvoGNN in modeling co-evolutionary patterns regardless of initial node embeddings. To sum up, the proposed EvoGNN-Evo learns node hidden representations from the co-evolution of node attribute and graph structure, and performs the best for predicting future attribute values and links.

More experimental results including tables and our analysis can be found in the Supplementary Materials.

Sensitivity and efficiency

Table 3 and Figure 3a shows EvoGNN's performance against different values of the network depth K from 1 to 5. We find that on $\mathcal{D}^{(2K)}$, $K = 3$ gives the best overall performance. On $\mathcal{D}^{(10K)}$, $K = 3$ and $K = 2$ have been able to give comparable good performance. In practice, we also find the choice of K has bigger influence on the performance of link prediction than the performance of attribute inference.



(a) Network depth K (b) Training range length T

(c) Efficiency test

Figure 3: Evaluating the effectiveness (F1) and efficiency (running time) of EvoGNN (the EG-EG model) under different values of parameters such as K and T.

Figure 3b shows EvoGNN's F1 score against different length of training range T. It is evident that including longer training range can result in better performance of EvoGNN. Figure 3c shows EvoGNN's running time per epoch against the network depth K – very close to a linear relationship.

Conclusion

In this work, we proposed a new framework for learning node embeddings from evolutionary attributed graph and inferring future node representations. It aggregated the information in previous snapshots to the current one by employing a multi-task loss function based on attribute inference and link prediction over time. Experimental results demonstrated that our model outperformed strong baselines on forecasting an entire future snapshot of personal attributes and co-authorship links in the evolutionary graphs.

References

Bruna, J.; Zaremba, W.; Szlam, A.; and LeCun, Y. 2013. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203*.

Coleman, J. S., and Coleman, J. S. 1994. *Foundations of social theory*. Harvard university press.

Defferrard, M.; Bresson, X.; and Vandergheynst, P. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems*, 3844–3852.

Févotte, C., and Idier, J. 2011. Algorithms for nonnegative matrix factorization with the β-divergence. *Neural computation* 23(9):2421–2456.

Grover, A., and Leskovec, J. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 855–864. ACM.

Hamilton, W.; Ying, Z.; and Leskovec, J. 2017a. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 1024–1034.

Hamilton, W. L.; Ying, R.; and Leskovec, J. 2017b. Representation learning on graphs: Methods and applications. *arXiv preprint arXiv:1709.05584*.

Henaff, M.; Bruna, J.; and LeCun, Y. 2015. Deep convolutional networks on graph-structured data. *arXiv preprint arXiv:1506.05163*.

Hochreiter, S., and Schmidhuber, J. 1997. Long short-term memory. *Neural computation* 9(8):1735–1780.

Kipf, T. N., and Welling, M. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.

Li, Y.; Yu, R.; Shahabi, C.; and Liu, Y. 2017. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. *arXiv preprint arXiv:1707.01926*.

Manessi, F.; Rozza, A.; and Manzo, M. 2017. Dynamic graph convolutional networks. *arXiv preprint arXiv:1704.06199*.

Nguyen, G. H.; Lee, J. B.; Rossi, R. A.; Ahmed, N. K.; Koh, E.; and Kim, S. 2018. Continuous-time dynamic network embeddings. In *Companion of the The Web Conference 2018 on The Web Conference 2018*, 969–976. International World Wide Web Conferences Steering Committee.

Niepert, M.; Ahmed, M.; and Kutzkov, K. 2016. Learning convolutional neural networks for graphs. In *International conference on machine learning*, 2014–2023.

Pareja, A.; Domeniconi, G.; Chen, J.; Ma, T.; Suzumura, T.; Kanezashi, H.; Kaler, T.; and Leisersen, C. E. 2019. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. *arXiv preprint arXiv:1902.10191*.

Qu, M.; Bengio, Y.; and Tang, J. 2019. Gmnn: Graph markov neural networks. In *Proceedings of the 36th International Conference on Machine Learning*.

Sankar, A.; Wu, Y.; Gou, L.; Zhang, W.; and Yang, H. 2018. Dynamic graph representation learning via self-attention networks. *arXiv preprint arXiv:1812.09430*.

Schlichtkrull, M.; Kipf, T. N.; Bloem, P.; Van Den Berg, R.; Titov, I.; and Welling, M. 2018. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, 593–607. Springer.

Seo, Y.; Defferrard, M.; Vandergheynst, P.; and Bresson, X. 2018. Structured sequence modeling with graph convolutional recurrent networks. In *International Conference on Neural Information Processing*, 362–373. Springer.

Xu, K.; Hu, W.; Leskovec, J.; and Jegelka, S. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*.

Ying, R.; He, R.; Chen, K.; Eksombatchai, P.; Hamilton, W. L.; and Leskovec, J. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 974–983. ACM.

Yu, B.; Yin, H.; and Zhu, Z. 2017. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *arXiv preprint arXiv:1709.04875*.

Zhou, L.; Yang, Y.; Ren, X.; Wu, F.; and Zhuang, Y. 2018. Dynamic network embedding by modeling triadic closure process. In *Thirty-Second AAAI Conference on Artificial Intelligence*.

Supplemental Materials of Evolutionary Graph Neural Networks

Related Work

Graph neural networks. Another recent work LRGCN (Li et al. 2019) was proposed for representation learning in time-evolving graphs. Specifically, it first generates node embeddings by using an architecture combining the RNN and the GCN, then it employs a self-attention mechanism for generating path embeddings of arbitrary length for the task of path classification (failure or not). Similar to our method EvoGNN, the input into this model also includes all historical graph snapshots. In addition, it also needs the path structure and label, which are highly traffic domain specific, for supervised training. This method cannot be directly applied in unsupervised manner such as modeling evolutionary co-authorship network.

Besides, a recent work JODIE (Kumar, Zhang, and Leskovec 2019) aimed at learning the trajectory of embeddings in dynamic bipartite graph of users and items. It was designed to model the temporal interactions between two parts of the dynamic bipartite graph where each interaction is marked by a timestamp and characterized by a feature vector. Thus, this method cannot be directly applied on general graph for capturing the co-evolutionary patterns.

Graph evolution analysis. There exists a line of work study the fundamental dynamics driving the evolution of graphs. The phenomenon of densification and shrinking diameters during graph evolution was studied in (Leskovec, Kleinberg, and Faloutsos 2007). A frequency-based pattern describing the evolution of large networks over time was proposed in (Berlingerio et al. 2009). In (Bringmann et al. 2010), the authors searched for typical patterns of structural changes in dynamic networks for predicting the evolution of social networks. (Kumar, Novak, and Tomkins 2010) studied the structure and evolution of online social networks. A evolutionary computation model for graph classification was proposed in (Jin, Young, and Wang 2010). (Bogdanov, Mongiovì, and Singh 2011) studied mining heavy subgraphs in time-evolving networks. The cohesive co-evolution patterns in dynamic attributed graphs was studied in (Desmier et al. 2012), and the trend mining in dynamic attributed graphs was studied in (Desmier et al. 2013). A comprehensive survey on evolutionary network analysis was provided in (Aggarwal and Subbian 2014).

Data Description

In table 1, we present the details on each graph snapshot in the $\mathcal{D}^{(2K)}$ dataset and the $\mathcal{D}^{(10K)}$ dataset.

Copyright © 2020, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Table 1: Evolutionary co-authorship datasets over 10 years.

	$\mathcal{D}^{(2K)}$				$\mathcal{D}^{(10K)}$			
Year	$\lvert\mathcal{V}\rvert$	$\lvert\mathcal{E}\rvert$	#V.	#W.	$\lvert\mathcal{V}\rvert$	$\lvert\mathcal{E}\rvert$	#V.	#W.
2001	874	1633	316	3549	4325	8080	448	6442
2002	1041	2390	251	2764	5235	11330	364	5253
2003	1211	3049	293	2945	6013	14632	414	5411
2004	1344	3847	262	3125	6873	18050	375	5702
2005	1405	4187	293	3172	7328	20752	424	5757
2006	1425	4182	266	3178	7582	21592	389	5826
2007	1467	4563	292	3245	7659	22690	420	5921
2008	1482	4705	267	3282	7707	22705	396	5937
2009	1452	4689	283	3283	7386	21924	412	5934
2010	1406	3907	265	3246	7146	19992	382	5888
Total	1928		316	3549	9854		448	6442

Experimental Settings

In this section, we present more details on baseline methods and training settings.

Baseline methods

- GraphSAGE: We test through 3 variants and report the best performance. They are separately trained on: (i) the most recent graph snapshot $(G^{t_{test}-1}, \mathbf{X}^{t_{test}-1})$, (ii) K most recent graph snapshots $(G^{t_{test}-1}, \mathbf{X}^{t_{test}-1}), \ldots, (G^{t_{test}-K}, \mathbf{X}^{t_{test}-K})$, and (iii) all the graph snapshots before t_{test}. Then each one was applied for inferring node embeddings at t_{test};
- DynamicTriad: This dynamic network embedding method only model the evolutionary pattern of graph structure and cannot handle node attribute. All graphs $G^{t_{test}-1}, G^{t_{test}-2}, \ldots, G^0$ before t_{test} are fed for training, and we focus on the task of *future graph link prediction* on $G^{t_{test}}$.
- DySAT: The treatment to this dynamic network embedding method is similar to DynamicTriad except that we use $\mathbf{X}^0$ for initializing the input node embeddings.
- DCRNN: For training, we use all node attribute matrices $\mathbf{X}^{t_{test}-1}, \mathbf{X}^{t_{test}-2}, \ldots, \mathbf{X}^0$ and the most recent graph $G^{t_{test}-1}$. The final prediction matrix generated by the model at t_{test} is directly used for the *future node attribute prediction* task, and the node embeddings outputted by the diffusion convolutional layer are used for the *future graph link prediction* task.
- STGCN: The treatment is similar to DCRNN except that the node embeddings outputted by the last spatio-temporal convolutional block are used for the *future graph link prediction* task.
- EvolveGCN: The input into this model includes all graph snapshots $(G^{t_{test}-1}, \mathbf{X}^{t_{test}-1}), \ldots, (G^0, \mathbf{X}^0)$ before t_{test}. We use the link prediction loss for training the model, and use the node embeddings outputted by

Table 2: Performance of EvoGNN on with different values of training length T.

	$\mathcal{D}^{(2K)}$					$\mathcal{D}^{(10K)}$				
	Attributes $\mathbf{X}^{2010}$		Links in G^{2010}			Attributes $\mathbf{X}^{2010}$		Links in G^{2010}		
Training	MAE	RMSE	AUC	F1	P@50, 100, 200	MAE	RMSE	AUC	F1	P@50, 100, 200
$01-09$	**0.451**	0.947	**0.142**	**0.265**	**0.80**, **0.83**, 0.70	**0.421**	**0.840**	**0.076**	**0.167**	**0.81**, **0.84**, 0.62
$02-09$	**0.451**	**0.945**	0.133	0.259	0.72, 0.80, **0.73**	0.422	0.845	0.070	0.148	0.76, 0.76, **0.72**
$03-09$	**0.451**	0.946	0.130	0.253	0.64, 0.66, 0.63	0.423	0.846	0.067	0.135	0.74, 0.72, 0.70
$04-09$	0.452	0.950	0.120	0.248	0.64, 0.56, 0.56	0.427	0.850	0.062	0.128	0.66, 0.70, 0.68
$05-09$	0.452	0.951	0.113	0.230	0.66, 0.63, 0.60	0.426	0.853	0.060	0.111	0.52, 0.58, 0.68
$06-09$	0.454	0.956	0.095	0.201	0.50, 0.62, 0.65	0.430	0.860	0.054	0.095	0.50, 0.47, 0.54
$07-09$	0.461	0.960	0.075	0.172	0.48, 0.62, 0.63	0.436	0.861	0.051	0.087	0.40, 0.42, 0.46
$08-09$	0.467	0.969	0.069	0.169	0.41, 0.60, 0.56	0.445	0.872	0.038	0.075	0.34, 0.38, 0.40

the evolving graph convolution unit for the task of *future graph link prediction*.

For GraphSAGE, since the input should be a single graph snapshot, we merge all K, or T, graph snapshots as a single one for the later 2 variants and fed it into the corresponding variant. Besides, for GraphSAGE and EvolveGCN, they do not have a transformation from the hidden embedding space to the raw attribute space, we compute the attribute inference matrix as $\mathbf{H}^{t_{test}} \cdot (\mathrm{tr}(\mathbf{H}^{t_{train}}) \cdot \mathbf{X}^{t_{train}})$ where $\mathrm{tr}(\,)$ denotes transpose operator. The last node embeddings matrix $\mathbf{H}^{t_{test}-1}$ from training graph snapshots and the last training attribute matrix $\mathbf{X}^{t_{test}-1}$ are treated as $\mathbf{H}^{t_{train}}$ and $\mathbf{X}^{t_{train}}$ for EvolveGCN. Since our evaluation tasks are performed on a future graph, static methods in transductive nature or dynamic methods with no forecasting capability cannot be directly applied.

Parameter and training settings

The core hyper-parameters of baseline methods are selected using the grid search strategy: triad closure process weight β_0 and temporal smoothness β_1 for DynamicTriad are found in $\{0.01, 0.1, 1, 10\}$; negative sampling ratio w_n for DySAT is found in $\{0.01, 0.1, 1\}$; the number of recurrent layers for DCRNN and the number of spatio-temporal convolutional blocks for STGCN are found in $\{1, 2\}$. The default summarize function is used for EvolveGCN. Other hyper-parameters are set following the recommended guidelines provided by authors. Because the raw attribute vectors are highly sparse, for all methods, we subsample the same number of close-to-zero predictions as the number of bigger-than-1 predictions for test to make it comparable.

Experimental results

The performance of EvoGNN with different values of training length T is presented in Table 2. On the $\mathcal{D}^{(2K)}$ dataset, EvoGNN achieves best performance for predicting links in G^{2010} when using all historical graph snapshots from 2001 to 2009; and has comparable good performance for predicting attributes in $\mathbf{X}^{2010}$ if being fed more snapshots starting from 2003. On the $\mathcal{D}^{(10K)}$ dataset, the trend is more evident, EvoGNN achieves best performance for both tasks when using all historical graph snapshots. Longer training ranges can lead to better performance of EvoGNN.

References

Aggarwal, C., and Subbian, K. 2014. Evolutionary network analysis: A survey. *ACM Computing Surveys (CSUR)* 47(1):10.

Berlingerio, M.; Bonchi, F.; Bringmann, B.; and Gionis, A. 2009. Mining graph evolution rules. In *joint European conference on machine learning and knowledge discovery in databases*, 115–130. Springer.

Bogdanov, P.; Mongiovì, M.; and Singh, A. K. 2011. Mining heavy subgraphs in time-evolving networks. In *2011 IEEE 11th International Conference on Data Mining*, 81–90. IEEE.

Bringmann, B.; Berlingerio, M.; Bonchi, F.; and Gionis, A. 2010. Learning and predicting the evolution of social networks. *IEEE Intelligent Systems* 25(4):26–35.

Desmier, E.; Plantevit, M.; Robardet, C.; and Boulicaut, J.-F. 2012. Cohesive co-evolution patterns in dynamic attributed graphs. In *International Conference on Discovery Science*, 110–124. Springer.

Desmier, E.; Plantevit, M.; Robardet, C.; and Boulicaut, J.-F. 2013. Trend mining in dynamic attributed graphs. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, 654–669. Springer.

Jin, N.; Young, C.; and Wang, W. 2010. Gaia: graph classification using evolutionary computation. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, 879–890. ACM.

Kumar, R.; Novak, J.; and Tomkins, A. 2010. Structure and evolution of online social networks. In *Link mining: models, algorithms, and applications*. Springer. 337–357.

Kumar, S.; Zhang, X.; and Leskovec, J. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1269–1278. ACM.

Leskovec, J.; Kleinberg, J.; and Faloutsos, C. 2007. Graph evolution: Densification and shrinking diameters. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 1(1):2.

Li, J.; Han, Z.; Cheng, H.; Su, J.; Wang, P.; Zhang, J.; and Pan, L. 2019. Predicting path failure in time-evolving graphs. *arXiv preprint arXiv:1905.03994*.